
Oliver Hauke

ist Mathematiker und IT-Referent im Kompetenzzentrum „Analyse und Auswertung“ des Statistischen Bundesamtes. Er verantwortet die Weiterentwicklung der technischen Infrastruktur des Forschungsdatenzentrums und berät das Netzwerk empirische Steuerforschung in der effizienten Nutzung der Systeme.

Annette Kristiansen

ist Ökonomin und Referentin im Referat „Unternehmenssteuern“ des Statistischen Bundesamtes. Sie beschäftigt sich mit der Zusammenführung der Unternehmenssteuerstatistiken und betreut das Netzwerk empirische Steuerforschung. Für ihre externe Promotion an der Universität Bremen untersucht sie die technologische Vielfalt multinationaler Unternehmen.

EFFIZIENTE ANALYSE GROSSER FORSCHUNGSDATEN MITHILFE VON R

Oliver Hauke, Annette Kristiansen

🔑 **Schlüsselwörter:** Effiziente Analysen, R-Programmierung, Forschungsdaten, Unternehmenssteuerstatistik, Business-Tax-Panel, Netzwerk empirische Steuerforschung

ZUSAMMENFASSUNG

Das Forschungsdatenzentrum des Statistischen Bundesamtes baut seine technische Infrastruktur gezielt aus, um der Wissenschaft erweiterte Analysemöglichkeiten amtlicher Daten bereitzustellen. Seit November 2025 steht Forschenden exklusiver Zugang zu diesen erweiterten Systemressourcen in R und Stata zur Verfügung. R-basierte Tools – insbesondere Apache Arrow, Parquet und DuckDB – ermöglichen es, gängige Analysen großer Forschungsdatensätze in Echtzeit auszuführen. Am Beispiel des Business-Tax-Panel (2013 bis 2019) werden die erzielbaren Effizienzgewinne bei der Auswertung umfangreicher steuerstatistischer Daten im Rahmen des Netzwerks empirische Steuerforschung demonstriert.

🔑 **Keywords:** *Efficient analysis – R programming – research data – corporate tax statistics – Business Tax Panel – empirical tax research network*

ABSTRACT

The Research Data Centre at the Federal Statistical Office is strategically expanding its technical infrastructure to provide researchers with enhanced analytical capabilities for official data. Since November 2025, researchers have had exclusive access to these extended system resources in R and Stata. R-based tools – particularly Apache Arrow, Parquet, and DuckDB – enable real-time analysis of large research datasets. Using the Business Tax Panel (2013–2019), this paper demonstrates the achievable efficiency gains in analysing large-scale tax statistics within the empirical tax research network.

5

Vergleich der Performanz

5.1 Benchmark-Setup

Um zwischen den in Abschnitt 2.3 vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Benchmarks sowie der vollständig reproduzierbare Code sind auf Open-code verfügbar¹.

Für die Untersuchung wurden simulierte Einzeldaten in verschiedenen Größen (0,1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP). Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahlen** (pro Statistik und Jahr),
2. **Regression** (Einflussfaktoren auf Verlustvorträge).

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

1. **Arbeitsspeicher**-Beanspruchung der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während längere Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant. Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet. Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend

¹www.opencode.de/URL-To-Be-Announced.

sind Aspekte wie einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

5.2 Baselines in SAS und Stata

Klassisch werden die betrachteten Auswertungsaufgaben im Statistischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten (etwa. 800.000 Beobachtungen) in SAS und Stata durchgeführt. Tabelle 1 (tba: Referenz zu Tabelle 1 fixen)

Die Ergebnisse zeigen, dass der umfangreiche Arbeitsspeicher des FDZ OnSite-Servers nicht ausreicht, um das ganze BTP in Stata einzulesen, sodass zwingend mit Variablenselektion gearbeitet werden muss. SAS zeigt seine Stärke anhand der geringen Beanspruchung des Arbeitsspeichers, benötigt aber mit etwa 2,3 Minuten pro Auswertung die doppelte Laufzeit gegenüber Stata.

5.3 Kleiner Vergleich von R zu den Baselines

Für 1% der simulierten BTP Daten, reduzieren die betrachteten Datenverarbeitungsmethode in R alle Zielgrößen gegenüber SAS und Stata erheblich. Selbst klassische Methoden in R sind mind. um den Faktor 7 schneller und benötigen nur 33 % des Festplattenspeichers. {polars}, {duckdb} und {arrow} lösen die betrachteten Aufgaben sogar

- > in **unter 2 Sekunden** (27-fach schneller),
- > mit **unter 300 MB Arbeitsspeicher**, {duckdb} und {arrow} sogar mit **unter 5 MB** (10-fach weniger)^{2,3}
- > und **5 GB Festplattenspeicher** (6-fach weniger gegenüber SAS und Stata).

Unter den klassischen R-Methoden bietet {data.table} die beste Performanz. Dabei sollte direkt bei der Einlese-

²Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hochperformanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

³Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hochperformanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

⁴{arrow} benötigt mit 274 MB deutlich mehr Arbeitsspeicher als {polars} und {duckdb}, aber kommt dennoch mit minimalen Entwicklungsumgebungen aus, bspw. einem standardmäßigen Laptop.

Tabelle 1

Baselines in SAS und Stata (anhand 1% des simuliertes BTP)

Software	Anwendung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
SAS	Fallzahl	65 MB	2,32 Min.	33 GB
	Regression	tba	2,40 Min.	33 GB
Stata	Fallzahl	33 GB	55,2 Sek.	32 GB
	Regression	33 GB	55,8 Sek.	32 GB

funktion `fread` auf die Auswahl der relevanten Variablen geachtet werden. Dadurch lassen sich unsere Auswertungen um einen Faktor 3 beschleunigen und der Arbeitsspeicher um einen Faktor über 1000 reduzieren.

Alle hoch-performanten Methoden sind dermaßen effizient, dass die Datenmenge von 800.000 Beobachtungen nicht mehr ausreicht, um zwischen den besten Methoden differenzieren zu können. Daher erhöhen wir im nächsten Experiment die Datenmenge auf 8.000.000 Beobachtungen.

5.4 Vergrößerung der Datenmenge

Anhand unserer Untersuchung (siehe Tabelle 4 und Tabelle 5), bietet `{arrow}+{parquet}` (bei Bedarf `+{duckdb}`) im Gesamtpaket die eindeutige besten Datenverarbeitungsmethoden für unsere Zwecke.

Kleinere Benchmarks:

- › Das geringe Arbeitsspeicherprofil Bedarf keiner weiteren Optimierung und ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, so dass die Methoden auf noch größere Datenmengen skalieren können. Ansonsten bietet `{duckdb}` und `{polars}` stärkere Optimierung des Arbeitsspeichers.
- › schnellste Laufzeit für große Datenmengen
- › das Parquet Dateiformat reduziert den benötigten Festplattenspeicher massiv.
- › beste Nutzererfahrung
 - › Leicht und verständlich - adaptiert nativ schöne `{tidyverse}` Syntax
 - › Flexibel, selektives Einlesen ist zwingend erforderlich
 - › hohe Stabilität, fehlende Features können durch leicht in `duckdb` oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- › kleiner Nachteil: Erfahrung notwendig, da erst Datenaufteilung/selektives Einlesen die maximale Perfor-

manz erreichen

Unerwarteterweise ist `{polars}` für das simulierte 10% BTP abgeschlagen (unter den schlechtesten Methoden) und für andere Datenmengen optimal schnell.

5.5 Erkenntnisse für kleine Datenmengen

- › Paket für die optimale Laufzeit nicht eindeutig
 - › 1e6: `arrow` > `duckdb` > `DT+SEL` > `polars`
 - › 1e5: `polars` > `duckdb` > `arrow` > `DT+SEL`
 - › 1e4: `polars` > `data.table` > `arrow` > `readr`
- › Dagegen ist das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet – sowohl in Speicherbedarf als auch in Laufzeit.
- › Datenaufteilung bietet nur high-performance Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.
- › Die Ergänzung von selektivem Einlesen ist immer vorteilhaft. High-Performance Methoden können aber bei einer aufgeteilten Datenhaltung darauf verzichten.
- › Von den klassischen Methoden ist `DT+SEL` am effizientesten, aber für unseren größten Anwendungsfall. Ein geschickter DTler kann mit moderaten Datenmengen auch sehr effizient arbeiten. Anfängern ist eher `{arrow}` zu empfehlen.
 - › 5x langsamer als `{arrow}`
 - › SEL notwendig, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
 - › SPLIT verschlechtert DT im Gegensatz zu `arrow`
 - › `{data.table}+csv` ist die beste "klassische" R-Methode und erreicht maximal 23.58s, in der größten Ordnung wie die nicht optimierte `Arrow`

Tabelle 2

Performanz Messungen für die Fallzahlberechnung anhand 1% des BTP. tba: mehr

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{polars}	Parquet	-	3,3 MB	1,0 Sek	4,9 GB
{duckdb}	Parquet	-	4,8 MB	1,5 Sek	4,9 GB
{arrow}	Parquet	Variablenauswahl	273,7 MB	1,7 Sek	4,9 GB
{data.table}	CSV	Variablenauswahl	42,9 MB	2,5 Sek	13,0 GB
{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,7 MB	2,8 Sek	4,9 GB
{arrow}	Parquet	-	272,8 MB	2,8 Sek	4,9 GB
{arrow}	Parquet	-	230,4 MB	3,8 Sek	4,9 GB
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	3,9 Sek	4,8 GB
{data.table}	CSV	-	33,7 GB	7,4 Sek	13,0 GB
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	50,4 GB	13,6 Sek	10,5 GB

Tabelle 3

Performanz Messungen für die Fallzahlberechnung anhand 1% des BTP. tba: mehr

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	529,7 MB	3,8 Sek	4,8 GB
{dplyr}	CSV	Variablenauswahl	154,8 MB	13,2 Sek	13,0 GB

Tabelle 4

Performanz Messungen für die Fallzahlberechnung anhand 10% des BTP.

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,9 MB	4,7 Sek	48,8 GB
{duckdb}	Parquet	-	4,7 MB	5,1 Sek	49,4 GB
{arrow}	Parquet	Variablenauswahl	702,1 MB	7,5 Sek	49,4 GB
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	4,7 MB	11,7 Sek	48,1 GB
{arrow}	Parquet	-	230,4 MB	17,0 Sek	49,4 GB
{arrow}	Parquet	-	701,2 MB	20,6 Sek	49,4 GB
{data.table}	CSV	Variablenauswahl	427,5 MB	23,6 Sek	130,2 GB
{polars}	Parquet	-	3,3 MB	1,2 Min	49,4 GB
{data.table}	CSV	-	336,0 GB	2,2 Min	130,2 GB
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	506,4 GB	5,0 Min	104,6 GB

Tabelle 5

Performanz Messungen für die Regressionsberechnung anhand 10% des BTP.

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	782,1 MB	5,2 Sek	48,1 GB
{dplyr}	CSV	Variablenauswahl	1,5 GB	1,9 Min	130,2 GB

Performance. Das die beste gemessene Performance für CSV (aufgeteilte Parquet sind doppelt so schnell). D.h. ein `{data.table}` Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat.

- › `base+readr` sind immer weit abgeschlagen

5.6 Ergebnisse

- › Für `{data.table}` ist die geschickte Programmierung noch viel wichtiger. Arrow wird höchstens langsam und beansprucht viel weniger RAM. Wohingegen `data.table` für den gesamten Datensatz (ohne Sel) das 600x an RAM belegt wie Arrow, damit viel schneller Abbricht und den schlechten Programmierer Abbrüche beschert.
 - › ohne Vorauswahl bricht DT ab, wenn das gesamte BTP verwendet werden soll. Arrow nur, wenn die Auswertung auch dazu führt, dass wirklich alle Variablen verarbeitet werden.
 - › mit DT lässt sich nicht so flexibel/schön arbeiten
- › Arrow verzeiht Fehler leichter als DT. Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir Count Ergebnisse in 11.73s, nur 5s über dem absoluten Optimum.
- › Wenn noch weniger RAM einsetzbar ist, benötigt Polars den wenigsten RAM, DuckDB nur geringfügig mehr

10 %: arrow + aufgeteilte parquet, duckdb, arrow variante
 1 %: polars + ein parquet, duckdb, arrow 0.1 %: polars + ein parquet, DT, arrow 0.01%: DT + csv, readr, polars

-> nicht eindeutig. Für kleinere Datenmengen mehr Auswahl und das vertraute auszuwählen. Für größere Datenmengen klare Empfehlung Arrow, w.g. einfacher Anwendung, Stabilität (ggü. Nutzerfehlern), Skalierbarkeit an die RAM-Grenze und darüber hinaus, höchste Performance für unsere Datenmengen+gute System Ressourcen.

-> für kleine Datenmengen macht chunking keinen Sinn.

Dateitypen:

Beide Anwendungen lassen sich in unter 20 Z. mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.

RAM Wachstum Arrow, Polars, DuckDB, `Data.table`, `readr`, `baser` mit und ohne `sel`.

512 im FDZ Obergrenze.

Gewinne ggü. Klassischen Methoden 7.X Min DT und sehr viel RAM im Default. Dennoch nicht komplett davon abraten.

5.7 Fazit (nach Hinten)

- › Empfehlung (insb. für unerfahrene Nutzende): Arrow, stabil, flexibel, einfach zu nutzen
- › Profi's in `data.table`, die sehr vertraut sind mit der Syntax, müssen nur in Randfällen wechseln. Falls die wirklich genutzten Variablenauswahl nicht durch den verfügbaren Arbeitsspeicher bedient werden können (dt brauchte ca. 3x RAM ggü. Festplatte).
- › neuste, beste, experimentell, bietet **polars** für moderate Datenmengen die effizienteste Variante
- › wer mit `sql` vertraut ist oder Features außerhalb des Scopes von Arrow braucht (auf großen Inputs), nutzt auch gerne `duckdb`

5.8 Anhang

Weitere Performancemessungen für kleinere Datenmengen.

->

Tabelle 6

Benchmarkergebnisse für Fallzahlen (kleine Stichproben des BTP)

Beob.	Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
10.000	{polars}	Parquet	-	3,3 MB	0,4 Sek	492,6 MB
10.000	{data.table}	CSV	Variablenauswahl	4,4 MB	0,7 Sek	1,3 GB
10.000	{arrow}	Parquet	Variablenauswahl	230,7 MB	1,1 Sek	492,6 MB
10.000	{readr}	CSV	Variablenauswahl	9,3 MB	1,2 Sek	1,3 GB
10.000	{arrow}	Parquet	-	229,8 MB	1,6 Sek	492,6 MB
10.000	{duckdb}	Parquet	-	4,7 MB	2,9 Sek	492,6 MB
10.000	{data.table}	CSV	-	3,4 GB	3,6 Sek	1,3 GB
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	4,2 Sek	486,9 MB
10.000	{arrow}	Parquet	-	230,4 MB	4,6 Sek	492,6 MB
10.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	5,1 GB	4,8 Sek	1,0 GB
10.000	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,9 MB	7,1 Sek	483,5 MB
10.000	{readr}	CSV	-	1,7 GB	8,6 Sek	1,3 GB
10.000	{arrow}	CSV	-	27,4 MB	15,7 Sek	1,3 GB
10.000	{data.table}	CSV.GZ	-	4,7 GB	21,2 Sek	421,9 MB
10.000	{base}	CSV	-	9,7 GB	4,9 Min	1,3 GB
1.000	{data.table}	CSV	Variablenauswahl	0,6 MB	0,1 Sek	129,8 MB
1.000	{readr}	CSV	Variablenauswahl	2,7 MB	0,2 Sek	129,8 MB
1.000	{polars}	Parquet	-	3,3 MB	0,4 Sek	48,1 MB
1.000	{data.table}	CSV	-	341,8 MB	0,9 Sek	129,8 MB
1.000	{duckdb}	Parquet	-	4,7 MB	1,1 Sek	48,1 MB
1.000	{arrow}	Parquet	Variablenauswahl	226,2 MB	1,2 Sek	48,1 MB
1.000	{arrow}	Parquet	-	225,3 MB	1,4 Sek	48,1 MB
1.000	{arrow}	CSV	-	3,5 MB	1,5 Sek	129,8 MB
1.000	{data.table}	CSV.GZ	-	491,7 MB	2,7 Sek	42,1 MB
1.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	510,9 MB	2,8 Sek	104,5 MB
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	2,9 Sek	53,5 MB
1.000	{arrow}	Parquet	-	230,4 MB	3,7 Sek	48,1 MB
1.000	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,7 MB	4,0 Sek	50,5 MB
1.000	{readr}	CSV	-	197,4 MB	4,9 Sek	129,8 MB
1.000	{base}	CSV	-	713,4 MB	23,3 Sek	129,8 MB

Tabelle 7

Benchmarkergebnisse für Regression (kleine Stichproben des BTP)

Beob.	Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
10.000	{dplyr}	CSV	Variablenauswahl	17,6 MB	1,3 Sek	1,3 GB
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	459,3 MB	5,0 Sek	486,9 MB
10.000	{dplyr}	CSV	-	1,8 GB	10,8 Sek	1,3 GB
1.000	{dplyr}	CSV	Variablenauswahl	4,2 MB	0,3 Sek	129,8 MB
1.000	{dplyr}	CSV	-	206,9 MB	6,4 Sek	129,8 MB
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	452,5 MB	6,8 Sek	53,5 MB